

Week-06

Backend Progress Report

Project Title: AI Campus Assistant

Course Title: Mobile Computing Lab
Course Code: CSE 4204

Submitted by:

Team name: CSE4204-8C-T05
Section: 8C

Name of group members	IDs
Gazi Jayed Rasul	11220320906
Mirajur Rahman	11220320917
Md. Shakil Ahammed	11220320920
Sheikh Tashreeful Islam (team-leader)	11220321042

Submitted to:

Md. Riaz Mahmud
Assistant Professor
Department of Computer Science and Engineering

Date of Submission: 29 June, 2026

Table of Contents

No.	Name	Page
1.	Team Information	1
2.	Project Overview	2
3.	Backend Technology Stack	3
4.	Database Design Summary	4 - 7
5.	Authentication Workflow	8
6.	Implemented APIs	9 - 10
7.	API Testing Results	11 - 16
8.	Current Development Progress	17 – 18
9.	Backend Folder Structure	19
10.	Github Repository	20
11.	Conclusion	21

Team Information

- **Team name:** CSE4204-8C-T05
- **Team Leader:** Sheikh Tashreeful Islam
- **Team Members:**

No.	Name	ID	Role
1	Gazi Jayed Rasul	11220320906	Database Manager
2	Mirajur Rahman	11220320917	Frontend Developer
3	Md. Shakil Ahammed	11220320920	AI Integration Lead
4	Sheikh Tashreeful Islam (team-leader)	11220321042	Backend Developer

- **Github Repository Link:** <https://github.com/TashreefulIslam/CSE4204-8C-T05-AI-Campus-Assistant>

Project Overview

AI Campus Assistant is a web-based academic management system developed to improve communication and academic activities within a university environment. The system provides a centralized platform where students, teachers, and administrators can efficiently access and manage academic information.

The primary objective of the project is to simplify common campus operations such as course management, assignment distribution, notice publishing, and AI-assisted academic support. By integrating these services into a single platform, the system aims to reduce manual processes, improve accessibility, and enhance the overall learning experience.

The backend of the system is developed using the **Laravel 12** framework following the RESTful API architecture. The application uses **MySQL** as its relational database and **Laravel Sanctum** for secure token-based authentication. During the current development phase, the backend has been configured with a normalized database structure, user authentication system, and several core API modules including User Authentication, Course Management, Assignment Management, and Notice Management etc.

The implemented backend establishes a scalable foundation that will support future frontend integration and additional modules such as course materials, class routines, examination schedules, CT notices, and AI-powered academic assistance.

Backend Technology Stack

The backend of the **AI Campus Assistant** was developed using modern web development technologies to ensure scalability, security, and maintainability. The following tools and technologies were used during the backend implementation.

Component	Technology Used	Purpose
Backend Framework	Laravel 12	Development of RESTful backend APIs and business logic
Programming Language	PHP 8.2	Server-side application development
Database Management System	MySQL	Storage and management of application data
Authentication	Laravel Sanctum	Token-based API authentication and authorization
API Architecture	RESTful API	Communication between frontend and backend
API Testing Tool	Postman	Testing API endpoints and validating responses
Local Development Environment	XAMPP	Running Apache, PHP and MySQL services locally
Dependency Manager	Composer	Installing and managing PHP packages
Version Control	Git & Github	Source code version management, collaboration with group members.
Code Editor	Visual Studio Code	Backend development and source code management

Database Design Summary

The database for the **AI Campus Assistant** was designed based on the approved Entity Relationship (ER) Diagram prepared during **Week-04**. The implementation follows a relational database model using **MySQL**, where primary keys, foreign keys, and relationships were established to maintain data consistency and integrity.

The database consists of ten main tables that support different functionalities of the system, including user management, course management, assignment handling, notice publishing, AI request logging, and academic scheduling. Appropriate one-to-many relationships were implemented using foreign key constraints to ensure proper communication between related entities.

Database Tables

Table Name	Purpose
users	Stores user information including students, teachers, and administrators with authentication details.
courses	Stores course information such as course code, course name, credit, and assigned teacher.
enrollments	Maintains the relationship between students and enrolled courses.
assignments	Stores assignment details including title, description, deadline, assigned course, and teacher.
ct_notices	Stores class test notices including exam date and examination schedule.
course_materials	Stores uploaded course materials and learning resources provided by teachers.
routines	Stores regular class routine information including day, time, and classroom.
exam_routines	Stores examination schedules for different courses.
notices	Stores university or departmental notices published by authorized users.
ai_requests	Stores prompts submitted to the AI assistant along with generated responses for future reference.

Database Relationships

The implemented database includes several important relationships to maintain data consistency:

- One teacher can teach multiple courses.
- One course can contain multiple assignments.
- One course can have multiple CT notices.
- One course can contain multiple course materials.
- One course can have one or more class routines.
- One course can have one or more examination schedules.
- One administrator can publish multiple notices.
- One user can submit multiple AI requests.
- Students can enroll in multiple courses through the enrollments table.

Database Implementation Status

- Database Created
- All Tables Implemented
- Primary Keys Configured
- Foreign Keys Implemented
- Table Relationships Established
- Laravel Migrations Executed Successfully



Figure 1: ER Diagram of the AI Campus Assistant Database

Table	Action	Rows	Type	Collation	Size	Overhead
☐ ai_requests	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ assignments	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ cache	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ cache_locks	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ courses	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ course_materials	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ ct_notices	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ enrollments	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ exam_routines	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ failed_jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ job_batches	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ migrations	★ Browse Structure Search Insert Empty Drop	14	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ notices	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ password_reset_tokens	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ personal_access_tokens	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
☐ routines	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ sessions	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-

Figure 2: phpMyAdmin showing the implemented database tables.

Authentication Workflow

The **AI Campus Assistant** uses Laravel Sanctum to implement secure token-based authentication for RESTful APIs. Sanctum provides a lightweight authentication mechanism that allows authenticated users to securely access protected API endpoints.

During backend development, the authentication module was implemented with user registration, login, logout, and authenticated profile retrieval functionalities. Passwords are securely hashed before being stored in the database, ensuring that plain-text passwords are never saved.

The authentication workflow consists of the following steps:

1. A new user registers by providing their name, email address, and password.
2. The system validates all input fields and checks for duplicate email addresses.
3. The user's password is automatically hashed using Laravel's built-in hashing mechanism before being stored in the database.
4. The user logs into the system using a valid email address and password.
5. If the credentials are valid, Laravel Sanctum generates a unique access token.
6. The client stores this access token and includes it as a Bearer Token in the Authorization header when accessing protected API endpoints.
7. Protected endpoints verify the token before processing the request.
8. During logout, the current access token is revoked, preventing further access using that token.

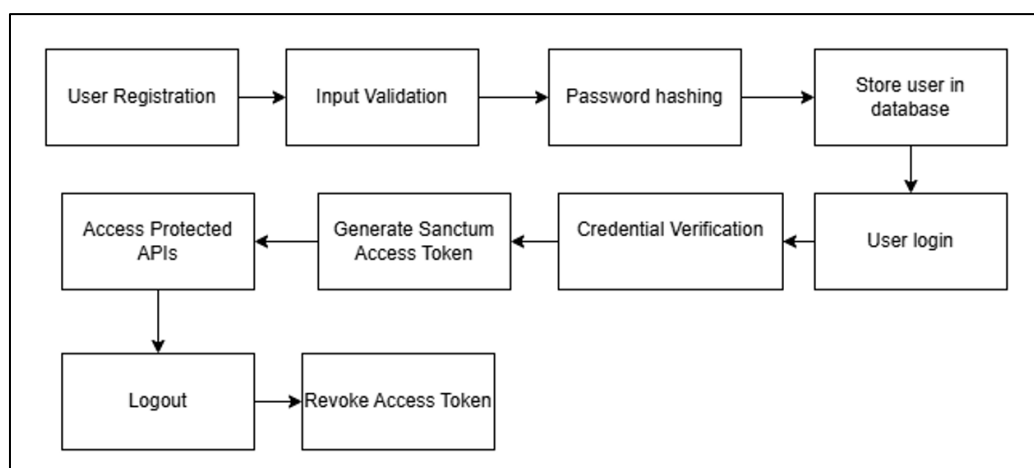


Figure 3: Authentication Workflow Diagram

Implemented APIs

The backend of the **AI Campus Assistant** follows the RESTful API architecture. During the Week-06 development phase, several core APIs were successfully implemented and tested using Postman. These APIs cover user authentication, course management, assignment management, and notice management.

1. Authentication APIs

Endpoint	Method	Description
/api/register	POST	Register a new user account
/api/login	POST	Authenticate user and generate Sanctum access token
/api/profile	GET	Retrieve authenticated user information
/api/logout	POST	Logout user and revoke current access token

2. Course Management APIs

Endpoint	Method	Description
/api/courses	POST	Create a new course
/api/courses	GET	Retrieve all courses
/api/courses/{id}	GET	Retrieve a specific course
/api/courses/{id}	PUT	Update course information
/api/courses/{id}	DELETE	Delete a course

3. Assignment Management APIs

Endpoint	Method	Description
/api/assignments	POST	Create a new assignment
/api/assignments	GET	Retrieve all assignments
/api/assignments/{id}	GET	Retrieve a specific assignment
/api/assignments/{id}	PUT	Update assignment information
/api/assignments/{id}	DELETE	Delete an assignment

4. Notice Management APIs

Endpoint	Method	Description
/api/notices	POST	Publish a new notice
/api/notices	GET	Retrieve all notices
/api/notices/{id}	GET	Retrieve a specific notice
/api/notices/{id}	PUT	Update notice information
/api/notices/{id}	DELETE	Delete a notice

Planned APIs for Future Development

The current backend development focuses on implementing the core functionalities required for the Week-06 milestone. Several additional modules have already been designed in the System Design Document and database schema but are scheduled for implementation in the upcoming development phases.

Planned Module	Current Status
CT Notice Management API	Planned
Course Material Management API	Planned
Routine Management API	Planned
Exam Routine Management API	Planned
AI Request Management API	Planned
Enrollment Management API	Planned

These modules will be implemented in subsequent development iterations while following the same RESTful API architecture and Laravel development standards established during the current phase.

API Testing Results

After implementing the backend APIs, comprehensive testing was performed using **Postman** to verify the correctness of each endpoint. The testing process included request validation, database operations, authentication, authorization, and error handling. The API responses were also verified through phpMyAdmin to ensure that database records were created, updated, and deleted correctly.

Testing Environment

Component	Tool Used
API Testing Tool	Postman
Backend Server	Laravel Development Server
Database Verification	phpMyAdmin
Authentication	Laravel Sanctum (Bearer Token)

API Testing Summary

Module	Tested Operations
Authentication	Register, Login, Profile, Logout
Course Management	Create, Retrieve All, Retrieve by ID, Update, Delete
Assignment Management	Create, Retrieve All, Retrieve by ID, Update, Delete
Notice Management	Create, Retrieve All, Retrieve by ID, Update, Delete

Validation and Error Handling

The following validation and error-handling scenarios were successfully tested:

- Duplicate email registration returns a validation error.
- Invalid login credentials return an authentication error.
- Required field validation prevents incomplete requests.
- Invalid resource IDs return appropriate error responses.
- Protected API endpoints require a valid Sanctum Bearer Token.
- Successful requests return appropriate HTTP status codes and JSON responses.

HTTP Status Codes Verified

Status Code	Description
200 OK	Request completed successfully
201 Created	Resource created successfully
401 Unauthorized	Authentication failed or invalid token
422 Unprocessable Entity	Validation failed due to invalid input

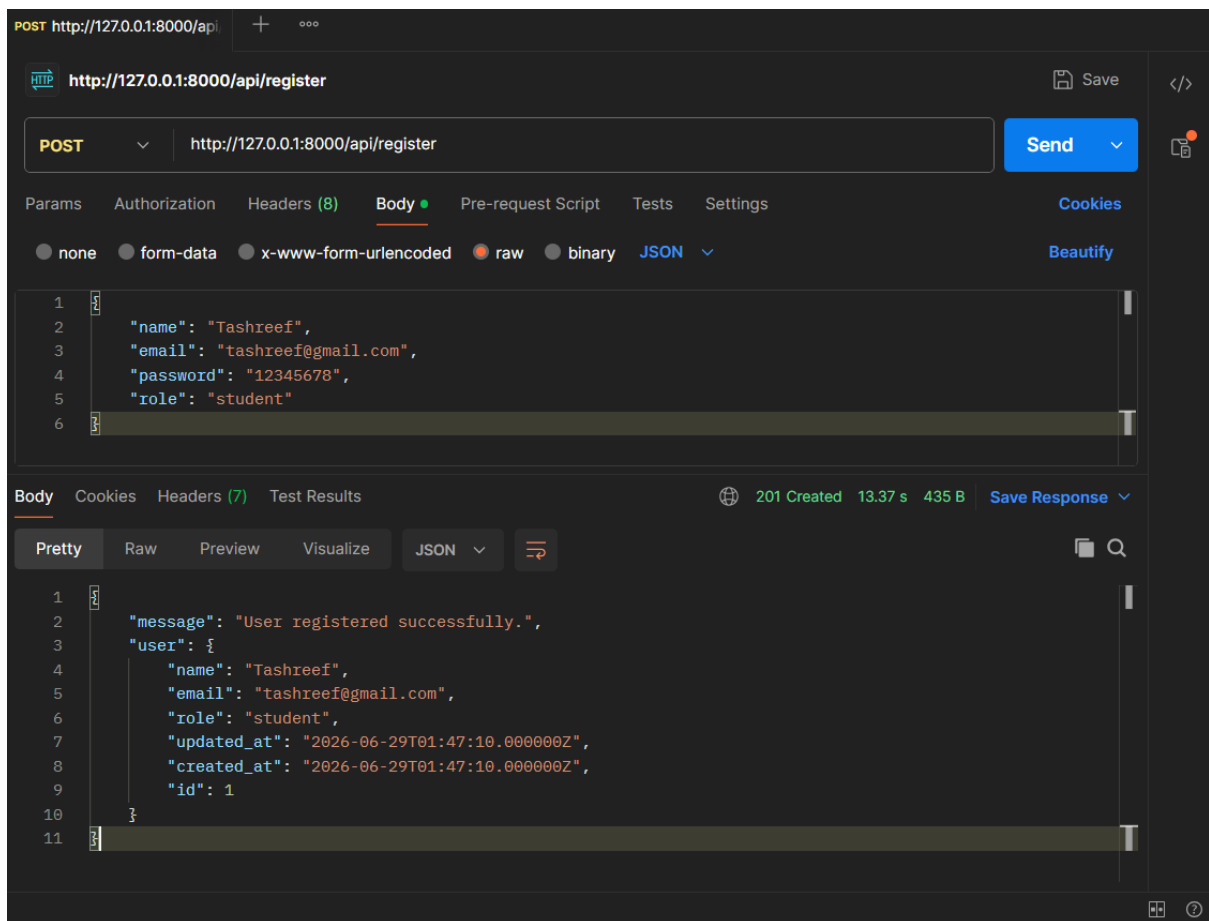


Figure 4: Successful User Registration (Postman)

	id	name	email	phone	password	role
 Edit  Copy  Delete	1	Tashreef	tashreef@gmail.com	NULL	\$2y\$12\$vxZmPXDDnTZ.zlaqrLn4DejWxg5jWEkd.9n0zH48EG3...	student

Figure 5: phpMyAdmin showing updated database records after API testing

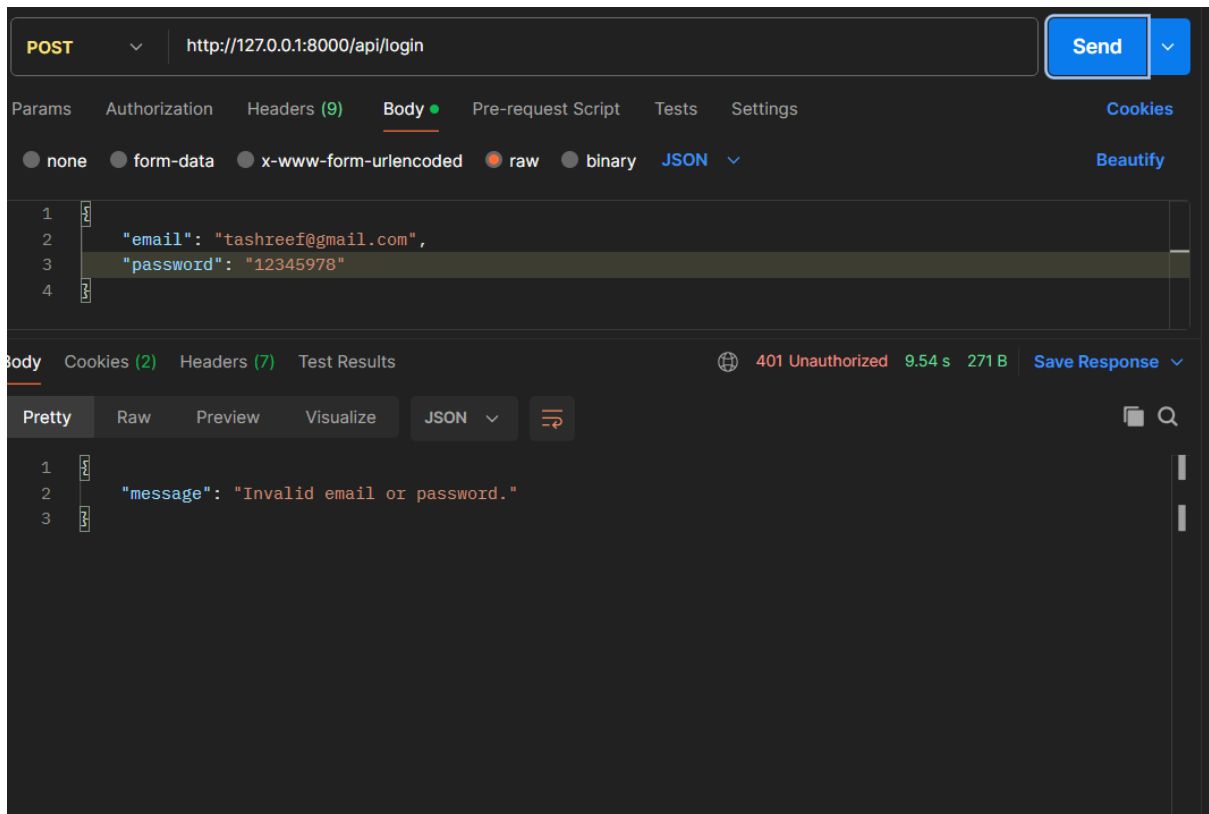


Figure 6: Login Test Using Invalid Password or Email

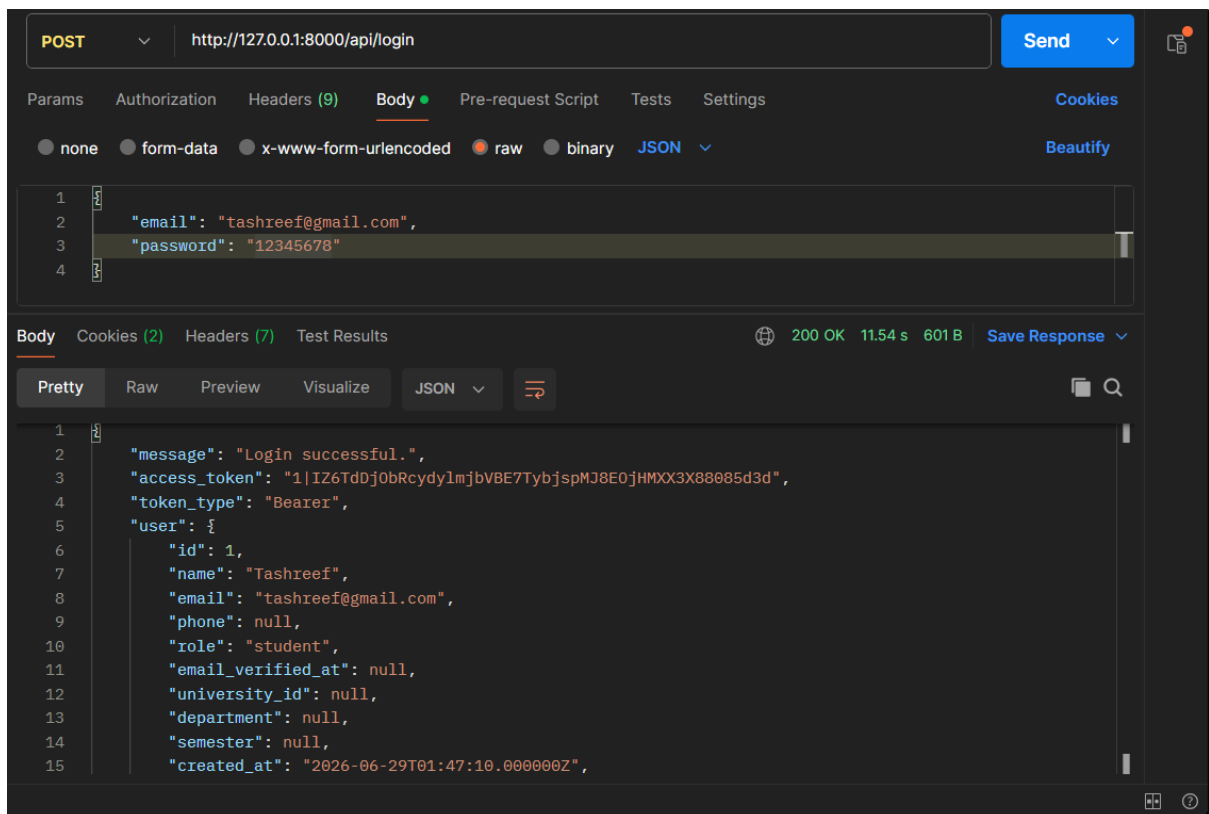


Figure 7: Successful User Login with Sanctum Token

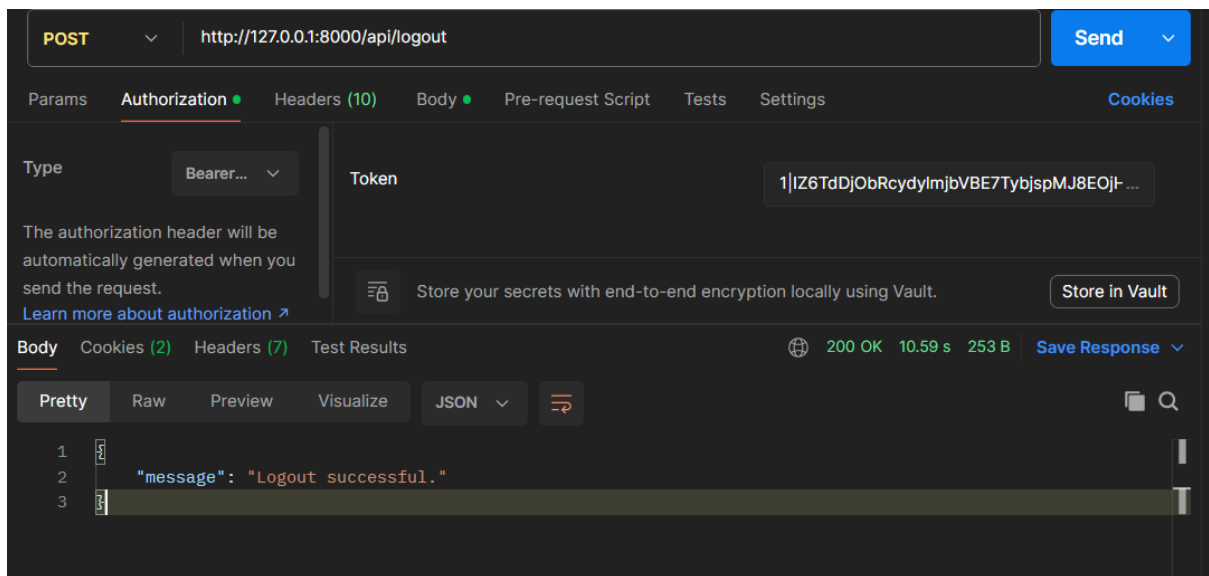


Figure 8: Logout API Testing

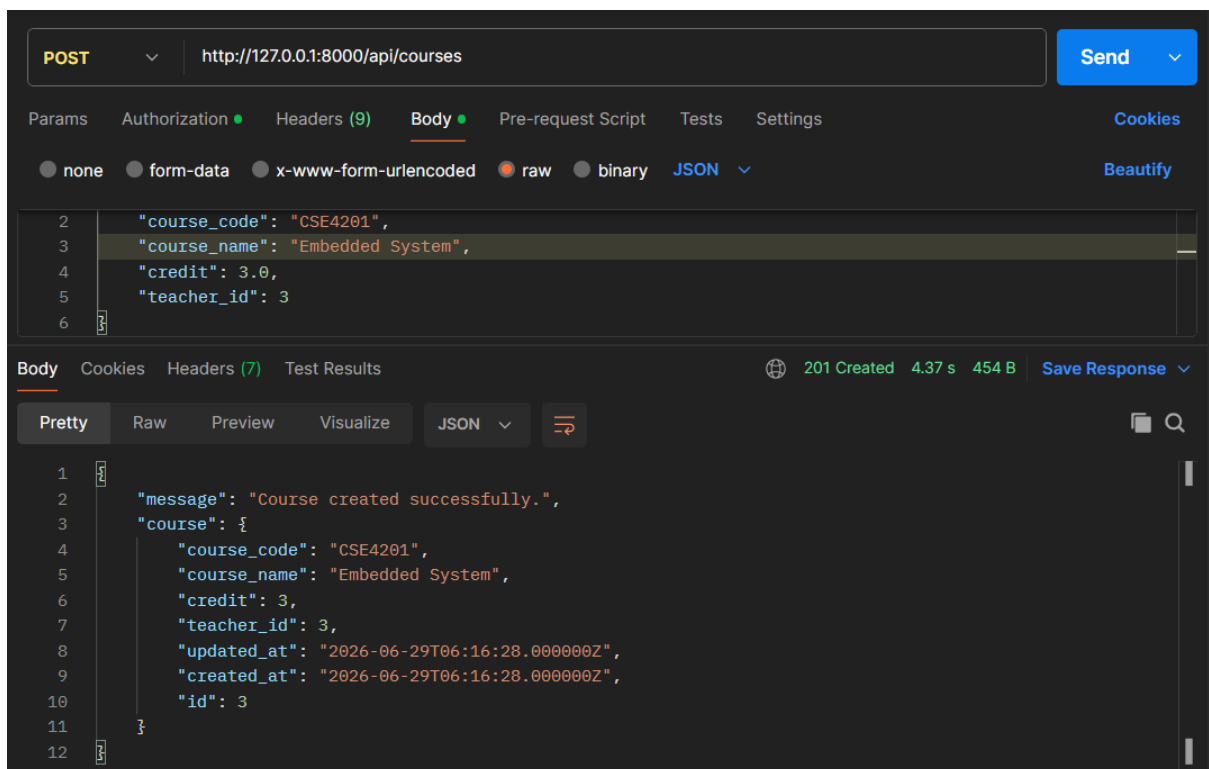


Figure 9: Course Create API Testing

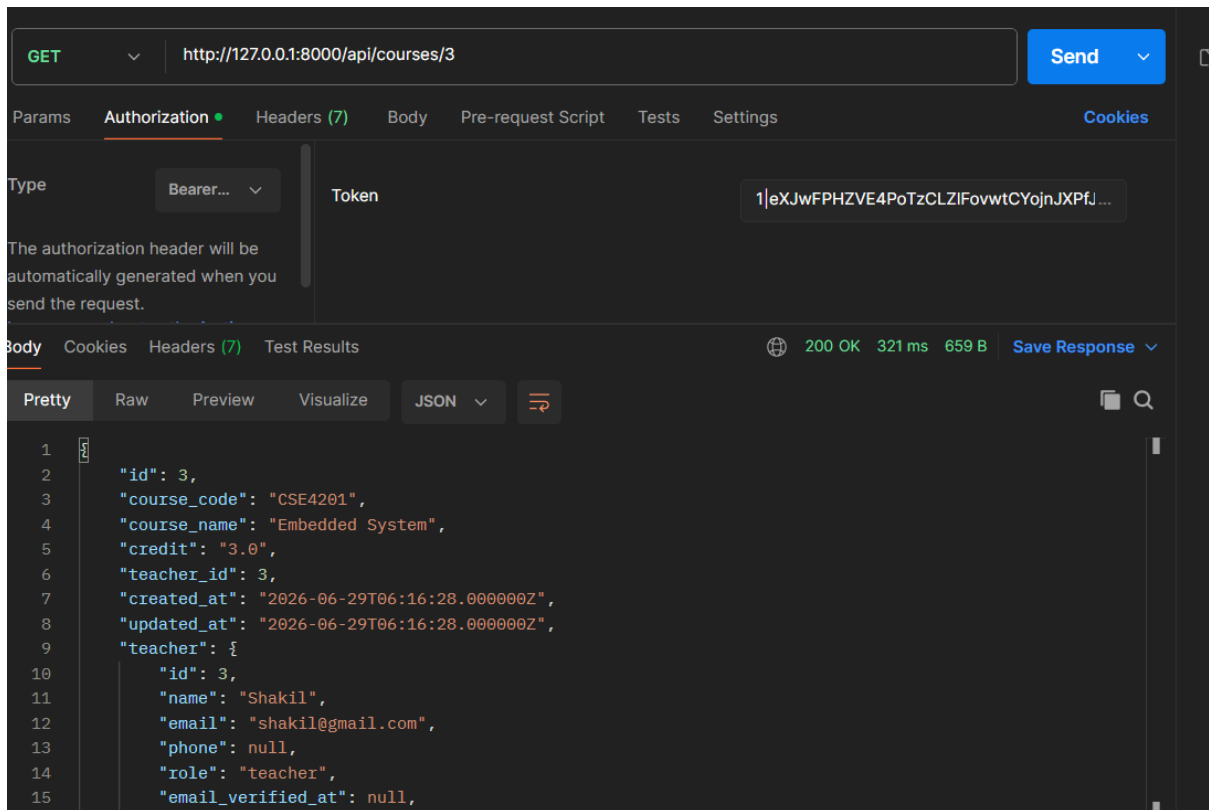


Figure 10: Course View API Testing (by using ID)

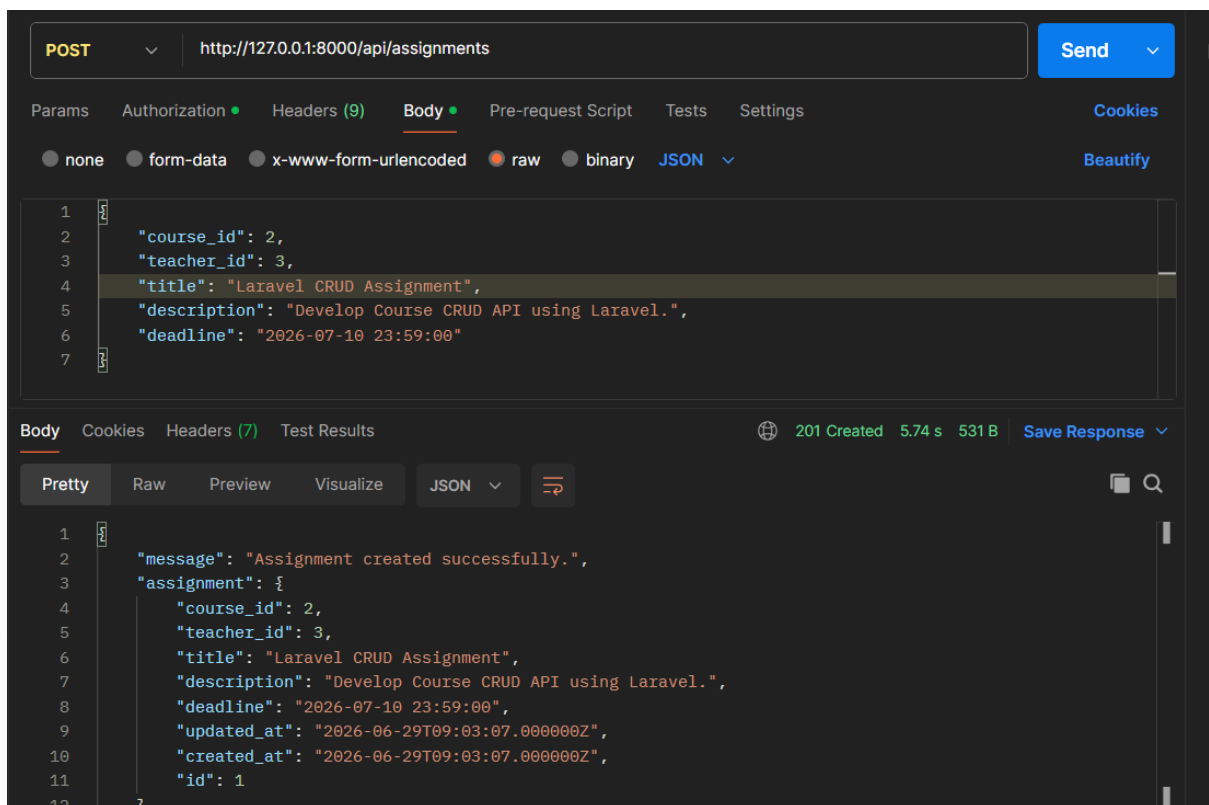


Figure 11: Assignment Create API Testing

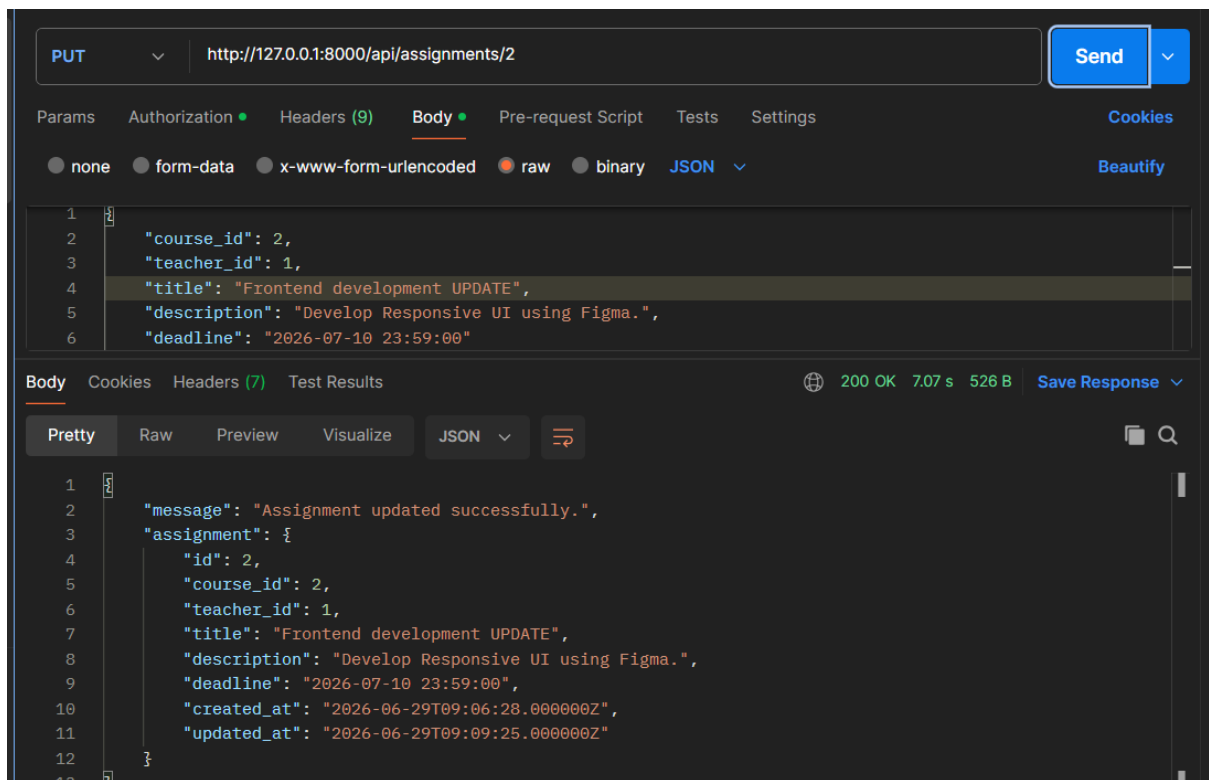


Figure 12: Assignment Update API Testing

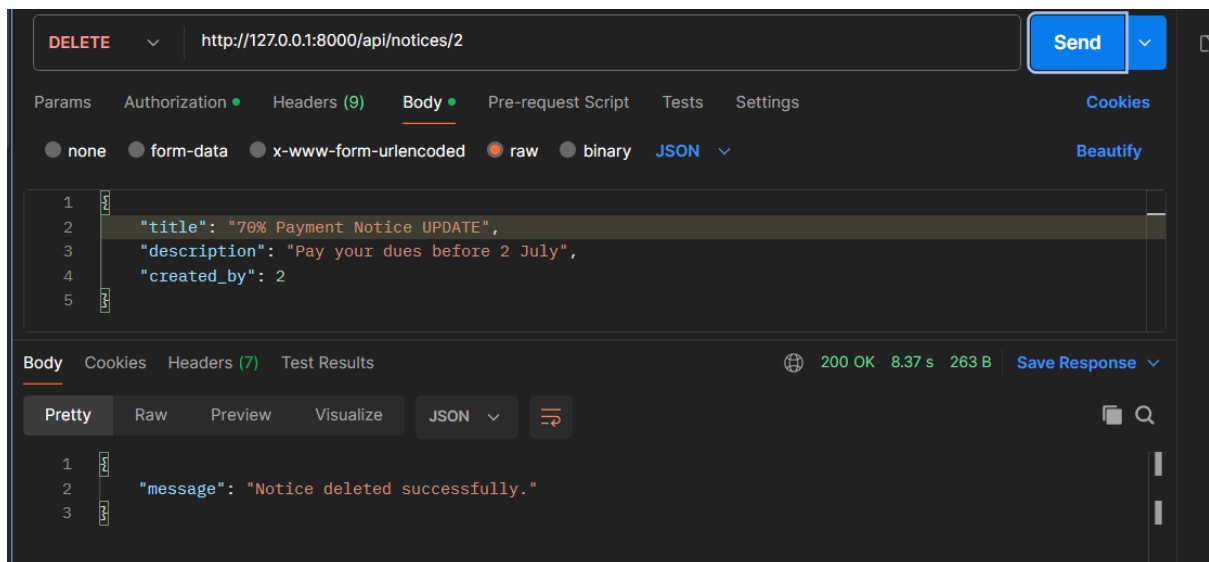


Figure 13: Notice Delete API Testing

Current Development Progress

The backend development of the **AI Campus Assistant** has successfully completed the core implementation planned for the Week-06 milestone. The project currently includes a functional database, secure authentication system, and several fully tested RESTful API modules.

Completed Development Tasks

Development Task	Status
Laravel Project Setup	Completed
MySQL Database Configuration	Completed
Database Schema Implementation	Completed
Laravel Migrations	Completed
Eloquent Models	Completed
Model Relationships	Completed
User Authentication	Completed
Laravel Sanctum Integration	Completed
Password Hashing	Completed
Request Validation	Completed
Error Handling	Completed
Course CRUD API	Completed
Assignment CRUD API	Completed
Notice CRUD API	Completed
API Testing using Postman	Completed
Database Verification using phpMyAdmin	Completed

Planned Development

The following modules have already been designed in the database schema and system design documents. Their implementation is planned for the next development phase:

- CT Notice Management
- Course Material Management
- Routine Management
- Exam Routine Management
- Enrollment Management
- AI Request Management
- Frontend Integration
- Role-Based Authorization
- AI Assistant Integration

The current backend architecture has been designed to support these modules without requiring major structural changes.

Backend Folder Structure

The backend project follows Laravel's standard directory structure, which promotes modularity, maintainability, and scalability. The application source code is organized into dedicated folders for controllers, models, routes, database migrations, middleware, configuration files, and other core components.

The project structure enables efficient management of backend resources and follows the MVC (Model–View–Controller) architectural pattern provided by the Laravel framework.

Important Project Directories

Directory	Purpose
app/Http/Controllers	Contains API controllers and business logic
app/Models	Contains Eloquent models and database relationships
database/migrations	Stores database migration files
routes	Defines API and web routes
config	Contains Laravel configuration files
storage	Stores application logs and generated files
vendor	Contains Composer-managed dependencies

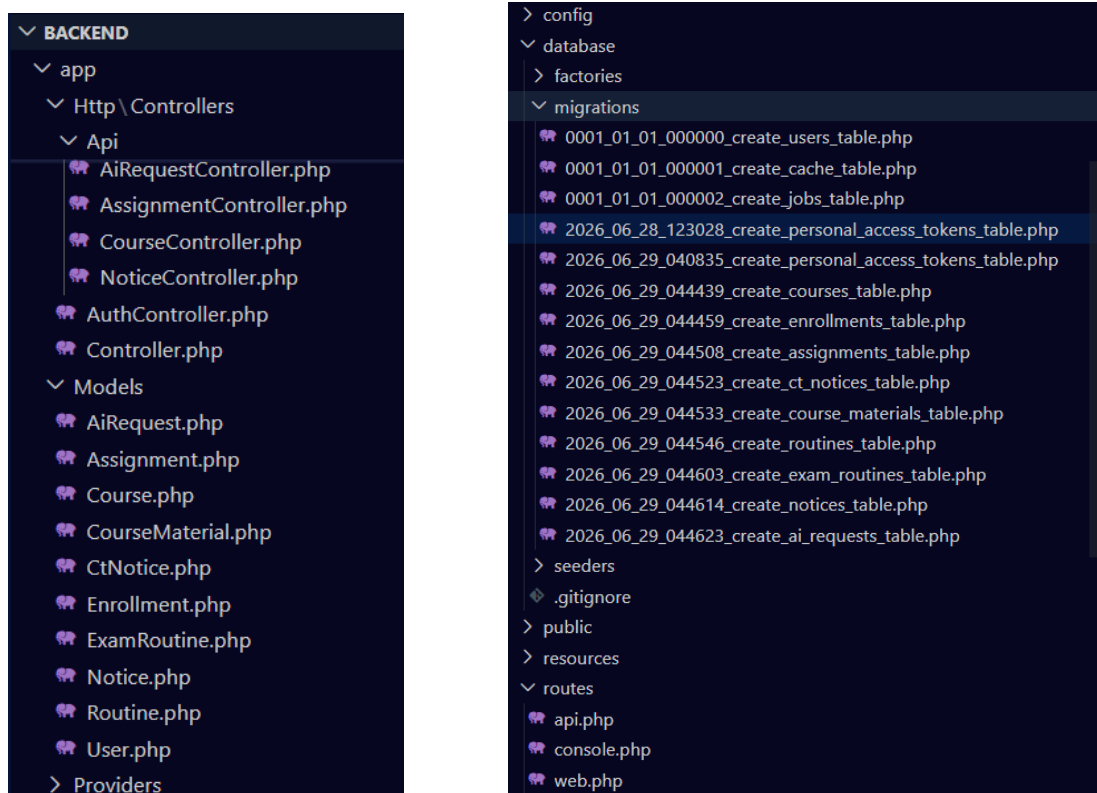


Figure 14: Visual Studio Code Explorer showing the Laravel backend project structure.

GitHub Repository

The backend source code of the **AI Campus Assistant** is maintained using **Git** and will be hosted on **GitHub** for version control, collaboration, and project management.

The repository contains the Laravel backend project, database migrations, models, controllers, API routes, configuration files, and project documentation. Git is used to maintain version history and support collaborative development among team members.

Repository Link:

<https://github.com/TashreefulIslam/CSE4204-8C-T05-AI-Campus-Assistant>

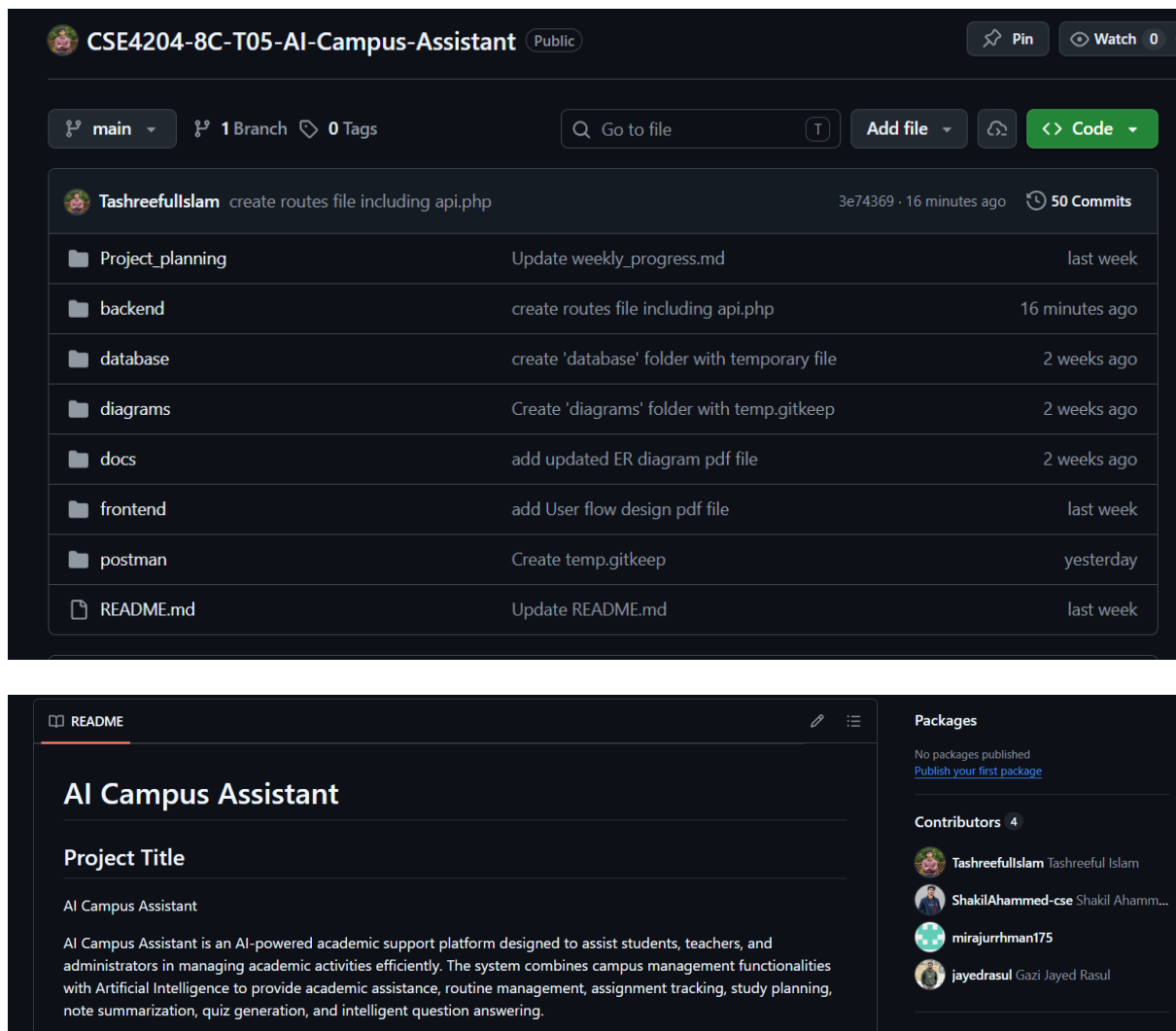


Figure 15: *GitHub repository homepage*

Conclusion

The Week-06 backend development milestone of the **AI Campus Assistant** has been successfully completed. During this phase, a secure and scalable backend was developed using **Laravel 12** and **MySQL**, following the RESTful API architecture.

The implemented backend includes a normalized relational database, user authentication using Laravel Sanctum, request validation, error handling, and fully functional CRUD APIs for Course Management, Assignment Management, and Notice Management. All implemented APIs were successfully tested using **Postman**, and database operations were verified through **phpMyAdmin**.

The current implementation provides a strong foundation for future development. In the upcoming phases, additional modules such as Course Materials, CT Notices, Class Routines, Exam Routines, AI Request Management, and frontend integration will be developed based on the existing architecture.

Overall, the backend developed during Week-06 satisfies the planned milestone objectives and establishes a maintainable, secure, and extensible architecture for the complete AI Campus Assistant system.